

**INTERNATIONAL CYBERSECURITY AND  
DIGITAL FORENSICS ACADEMY**

Nasarawa State, Nigeria

**LAB REPORT**

**Lab 03: Building Your First PLC Program**

| Item                | Details                                            |
|---------------------|----------------------------------------------------|
| Lab Title           | Lab 03: Building Your First PLC Program (TankFill) |
| Student Name        | Sunusi Abdullahi                                   |
| Student ID (Reg No) | C11/26/ACIS/17362                                  |
| Version             | Version 1.0                                        |

## 1. Executive Summary

---

This laboratory moved from reading PLC programs to building one. The task was to create a control program for an automatic liquid-receiving tank that is 0 to 10 feet deep. The tank has a fill valve, a level transmitter, an emergency-stop button, and a high-level beacon and horn.

Using OpenPLC Editor, I created a new Ladder Diagram project called TankFill. I defined ten variables, using BOOL for on/off signals and REAL for level values and setpoints, and built five ladder rungs. The program opens the fill valve when the level falls to 2 ft, keeps it open while the tank fills, and closes it at 8 ft. The valve also closes when the emergency stop is pressed or when the level reaches the 9 ft high alarm. Alarms switch on at 1 ft (low) and 9 ft (high).

The logic was checked with a test table of seven scenarios. The report also explains why the emergency stop must override automatic control: safety always comes before normal operation.

## 2. Lab Objectives

---

By the end of this lab, I should be able to:

- Create a new OpenPLC Ladder Diagram project and POU.
- Create PLC variables and use BOOL for on/off signals and REAL for analogue process values.
- Build power rails and ladder rungs using normal and negated contacts, comparison expressions, and SET and RESET coils.
- Build automatic tank-fill logic between the setpoints of 2 ft and 8 ft.
- Add emergency-stop protection and high-level and low-level alarm logic.
- Document and test the intended behaviour before connecting to an HMI or real equipment.

## 3. Tools and Resources Used

---

| Tool / Resource                             | What it was used for                                                                                            |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| ICDFA PLC Virtual Machine (IP 192.168.1.10) | The training PLC environment supplied by ICDFA. All work was done inside it.                                    |
| Virtualisation platform: (version)          | Used to run the VM from a clean snapshot on an isolated Host-Only network. Write the name and version you used. |
| OpenPLC Editor (version)                    | Used to create the TankFill project, the variables and the Ladder Diagram.                                      |
| TankFill project folder                     | The saved project files, which are submitted with this report.                                                  |
| Screenshot tool                             | Used to save evidence at each step.                                                                             |

### 3.1 The System (the problem)

An automatic tank-filling system has these devices:

| Device                                         | Type            | Job                                                         |
|------------------------------------------------|-----------------|-------------------------------------------------------------|
| Receiving tank, 0 to 10 ft                     | Process vessel  | Holds the liquid. Level range is 0 to 10 ft.                |
| Fill valve                                     | Discrete output | Opens or closes to let liquid into the tank.                |
| Level indicating transmitter (LIT), 4 to 20 mA | Analogue input  | Measures the level and sends it to the PLC.                 |
| Emergency-stop push button (latched)           | Discrete input  | When pressed, it stays pressed (latched) and stops filling. |
| High-level alarm beacon and horn               | Discrete output | Warn people that the tank is too full.                      |

#### Setpoints given in the lab:

| Setpoint       | Value  | Meaning                                            |
|----------------|--------|----------------------------------------------------|
| TankLoAlarm_SP | 1.0 ft | At or below this level the low-level alarm is ON.  |
| TankEmpty_SP   | 2.0 ft | At or below this level the automatic fill starts.  |
| TankFull_SP    | 8.0 ft | At or above this level the automatic fill stops.   |
| TankHiAlarm_SP | 9.0 ft | At or above this level the high-level alarm is ON. |

## 4. Methodology (Step-by-Step Procedure)

---

I followed the 14 steps of the lab brief in order. A screenshot was captured at each important step. The screenshots are numbered Figures 1 to 14.

### 4.2 The Steps

**Step 1: Start the VM and create the project.** I started the PLC VM from a clean snapshot on the isolated Host-Only network. I launched OpenPLC Editor and created a new project folder named **TankFill**.

**Step 11: Check the program.** I checked every variable name and data type. I checked that every contact, coil and expression is wired to a complete rung and that nothing is floating. My checklist:

| Check                                                                  | Result   | Done |
|------------------------------------------------------------------------|----------|------|
| All 10 variable names are spelled exactly as in the Variable Plan      | Yes / No | [ ]  |
| BOOL used for 5 discrete variables; REAL used for 5 analogue variables | Yes / No | [ ]  |
| Setpoints are 2.0, 8.0, 1.0 and 9.0                                    | Yes / No | [ ]  |
| Every rung runs from the left rail to the right rail                   | Yes / No | [ ]  |
| No contact, coil or expression is floating                             | Yes / No | [ ]  |

ICDFA-PLC-Engineering-v1.0 (LAB01-START) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Project: Unnamed, Res0

Class Filter: All

| #  | Name              | Class | Type | Location | Initial Value | Option | Documentation                                            |
|----|-------------------|-------|------|----------|---------------|--------|----------------------------------------------------------|
| 1  | TankFillAuto_Cmd  | Local | BOOL |          |               |        | Internal automatic fill latch/command                    |
| 2  | TankValveOpen     | Local | BOOL |          |               |        | Discrete command/status for the fill valve               |
| 3  | Estop_Cmd         | Local | BOOL |          |               |        | Emergency-stop input; TRUE forces automatic fill to stop |
| 4  | TankLevelHi_Alarm | Local | BOOL |          |               |        | High-level alarm state                                   |
| 5  | TankLevelLo_Alarm | Local | BOOL |          |               |        | Low-level alarm state                                    |
| 6  | TankLevel_LIT     | Local | REAL |          |               |        | Measured tank level in feet                              |
| 7  | TankEmpty_SP      | Local | REAL |          | 2.0           |        | Automatic fill start setpoint (2.0 ft)                   |
| 8  | TankFull_SP       | Local | REAL |          | 8.0           |        | Automatic fill stop setpoint (8.0 ft)                    |
| 9  | TankLoAlarm_SP    | Local | REAL |          | 1.0           |        | Low-level alarm setpoint (1.0 ft)                        |
| 10 | TankHiAlarm_SP    | Local | REAL |          | 9.0           |        | High-level alarm setpoint (9.0 ft)                       |

Search Console PLC Log

Library | Debugger

Search

- Standard function blocks
- Additional function blocks
- Arduino
- Microover
- Communication
- PIAM Modules
- Synergy Logic Modules
- MQTT
- Sequent Microsystems Modules
- Jaguar
- SL-RP4
- Type conversion
- Numerical
- Arithmetic
- Time
- Bit-shift
- Bitwise
- Selection
- Comparison
- Character string
- Native POUs
- User-defined POUs

2:27 PM 24/09/2026

ICDFA-PLC-Engineering-v1.0 (LAB01-START) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Project: Unnamed, Res0

Class Filter: All

| # | Name             | Class | Type | Location | Initial Value | Option | Documentation                              |
|---|------------------|-------|------|----------|---------------|--------|--------------------------------------------|
| 1 | TankFillAuto_Cmd | Local | BOOL |          |               |        | Internal automatic fill latch/command      |
| 2 | TankValveOpen    | Local | BOOL |          |               |        | Discrete command/status for the fill valve |

Search Console PLC Log

Library | Debugger

Search

- Standard function blocks
- Additional function blocks
- Arduino
- Microover
- Communication
- PIAM Modules
- Synergy Logic Modules
- MQTT
- Sequent Microsystems Modules
- Jaguar
- SL-RP4
- Type conversion
- Numerical
- Arithmetic
- Time
- Bit-shift
- Bitwise
- Selection
- Comparison
- Character string
- Native POUs
- User-defined POUs

2:37 PM 24/09/2026

## 5. Evidence and Analysis

### 5.1 Evidence 1: Variable / Tag Table

**Question:** Show the completed variable table (Figures 4 and 5) and explain the data types.

#### Part A: The variables

All variables were created as **Local** variables of the TankFill POU. In a real plant, Estop\_Cmd and TankLevel\_LIT would be linked to physical inputs, and TankValveOpen and the alarms to physical outputs. The name LIT means Level Indicating Transmitter.

| Variable          | Type | Initial value | Purpose                                                                     |
|-------------------|------|---------------|-----------------------------------------------------------------------------|
| TankFillAuto_Cmd  | BOOL | FALSE         | Internal automatic fill latch / command. TRUE means "filling is requested". |
| TankValveOpen     | BOOL | FALSE         | Command / status of the fill valve. TRUE = valve open.                      |
| Estop_Cmd         | BOOL | FALSE         | Emergency-stop input. TRUE forces the automatic fill to stop.               |
| TankLevelHi_Alarm | BOOL | FALSE         | High-level alarm state (beacon and horn).                                   |
| TankLevelLo_Alarm | BOOL | FALSE         | Low-level alarm state.                                                      |
| TankLevel_LIT     | REAL | 0.0           | Measured tank level in feet, from the 4 to 20 mA transmitter.               |
| TankEmpty_SP      | REAL | 2.0           | Automatic fill start setpoint (ft).                                         |
| TankFull_SP       | REAL | 8.0           | Automatic fill stop setpoint (ft).                                          |
| TankLoAlarm_SP    | REAL | 1.0           | Low-level alarm setpoint (ft).                                              |
| TankHiAlarm_SP    | REAL | 9.0           | High-level alarm setpoint (ft).                                             |

#### Part B: Why BOOL and REAL?

|              | BOOL (Boolean)                                                               | REAL                                                                               |
|--------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Values       | Only TRUE (1) or FALSE (0).                                                  | Numbers with a decimal point, for example 5.37.                                    |
| Used for     | Discrete on/off things: valve command, E-stop, alarms, latch.                | Analogue values: tank level and the setpoints in feet.                             |
| Why suitable | A valve is open or closed, and an alarm is on or off. Two states are enough. | Level changes smoothly, so we need decimals to compare with 2.0, 8.0, 1.0 and 9.0. |

#### Part C: Worked example (how the 4 to 20 mA signal becomes feet)

The transmitter sends 4 mA at 0 ft and 20 mA at 10 ft. What level is shown when the signal is 12 mA?

Step 1: Range of the signal =  $20 - 4 = 16$  mA.

Step 2: Signal above the start =  $12 - 4 = 8$  mA.

Step 3: Fraction of the range =  $8 / 16 = 0.5$ .

Step 4: Level =  $0.5 \times 10 = 5.0$  ft.

**Answer:** 12 mA means 5.0 ft. Using the same working, the setpoints are:

| Setpoint       | Level (ft) | Signal (mA)                        |
|----------------|------------|------------------------------------|
| TankLoAlarm_SP | 1.0        | $4 + (1 / 10 \times 16) = 5.6$ mA  |
| TankEmpty_SP   | 2.0        | $4 + (2 / 10 \times 16) = 7.2$ mA  |
| TankFull_SP    | 8.0        | $4 + (8 / 10 \times 16) = 16.8$ mA |
| TankHiAlarm_SP | 9.0        | $4 + (9 / 10 \times 16) = 18.4$ mA |

## 5.2 Evidence 2: The Ladder Program, Rung by Rung

**Question:** Show how the ladder logic was built and explain each rung. Screenshots are Figures 6 to 12.

**How to read a ladder:** Power flows from the left rail to the right rail. When a complete path is found, the coil at the end is energised. Rungs are solved from top to bottom in every scan of the PLC.

**Check on your screen:** In OpenPLC the comparison (for example TankLevel\_LIT <= TankEmpty\_SP) is entered as a variable expression or comparison block. Write here exactly how your editor let you enter it: \_\_\_\_\_.

### Rung 1: Valve control (Figure 7)

**Requirement:** The valve opens when automatic fill is requested and the high-level alarm is not active.

```
|      TankFillAuto_Cmd      TankLevelHi_Alarm      TankValveOpen  |  
|-----| |-----|/|----- ( )-----|
```

**Expression:** TankValveOpen = TankFillAuto\_Cmd AND NOT TankLevelHi\_Alarm

**Explanation:** The first contact is normally open, so it lets power pass only when TankFillAuto\_Cmd is TRUE. The second contact is negated (normally closed), so it lets power pass only when TankLevelHi\_Alarm is FALSE. If both allow power, the coil TankValveOpen is energised and the valve opens. If the high alarm turns ON, the negated contact opens and the valve closes at once.

### Rung 2: Automatic start (Figure 8)

**Requirement:** When the level falls to the empty setpoint (2.0 ft) or lower, start automatic filling.

```
|                                     TankFillAuto_Cmd  |  
|-----[ TankLevel_LIT <= TankEmpty_SP ]----- (S)-----|
```

**Expression:** IF TankLevel\_LIT <= TankEmpty\_SP THEN SET TankFillAuto\_Cmd

**Explanation:** The comparison is TRUE when the level is 2.0 ft or less. Then the SET coil (S) turns TankFillAuto\_Cmd ON. A SET coil latches: it stays ON even after the level rises above 2.0 ft. Only a RESET coil can turn it OFF. This is what keeps the valve open while the tank fills.

### Rung 3: Automatic stop and safety (Figure 9)

**Requirement:** Stop the automatic fill when the tank is full, or the emergency stop is pressed, or the high-level alarm is active.

```
|
|-----[ TankLevel_LIT >= TankFull_SP ]-----+----- TankFillAuto_Cmd |
|                                                     (R)-----|
| |
| +---| |--- Estop_Cmd -----+
| |
| +---| |--- TankLevelHi_Alarm -----+
|
```

**Expression:** IF (TankLevel\_LIT >= TankFull\_SP) OR Estop\_Cmd OR TankLevelHi\_Alarm THEN RESET TankFillAuto\_Cmd

**Explanation:** The three conditions are in parallel, so any one of them is enough (OR). When one is TRUE, power reaches the RESET coil (R) and TankFillAuto\_Cmd goes OFF. Then Rung 1 closes the valve. Rung 3 is placed after Rung 2 on purpose. The PLC solves rungs from top to bottom, so if Rung 2 sets the command and Rung 3 resets it in the same scan, the last action wins and the stop condition wins.

### Rung 4: High-level alarm (Figure 10)

**Requirement:** Switch on the high-level alarm when the level reaches 9.0 ft or more.

```
|
|-----[ TankLevel_LIT >= TankHiAlarm_SP ]----- ( )-----|
|                                                     TankLevelHi_Alarm |
```

**Expression:** TankLevelHi\_Alarm = (TankLevel\_LIT >= TankHiAlarm\_SP)

**Explanation:** When the level is 9.0 ft or more, the comparison is TRUE and the coil TankLevelHi\_Alarm is energised. This drives the beacon and horn. When the level drops below 9.0 ft, the alarm goes OFF by itself. This alarm also feeds Rung 1 and Rung 3, so it closes the valve and resets the fill command.

### Rung 5: Low-level alarm (Figure 11)

**Requirement:** Switch on the low-level alarm when the level falls to 1.0 ft or less.

```
|
|-----[ TankLevel_LIT <= TankLoAlarm_SP ]----- ( )-----|
|                                                     TankLevelLo_Alarm |
```

**Expression:** TankLevelLo\_Alarm = (TankLevel\_LIT <= TankLoAlarm\_SP)

**Explanation:** When the level is 1.0 ft or less, the coil TankLevelLo\_Alarm is energised. It warns the operator that the tank is nearly empty, which is lower than the 2.0 ft fill-start level. It goes OFF by itself when the level rises above 1.0 ft.

Key: | | = normally open contact, | / | = normally closed (negated) contact, [ ] = comparison, ( ) = coil, (S) = SET coil, (R) = RESET coil.

## Complete Ladder Diagram and How the Process Works (Figure 12)

Figure 12 shows the completed program. The table below is a summary of the five rungs.

| Rung | Job                       | Condition                                                                  | Action                     |
|------|---------------------------|----------------------------------------------------------------------------|----------------------------|
| 1    | Valve control             | TankFillAuto_Cmd AND NOT TankLevelHi_Alarm                                 | Energise TankValveOpen     |
| 2    | Automatic start           | TankLevel_LIT <= TankEmpty_SP (2.0 ft)                                     | SET TankFillAuto_Cmd       |
| 3    | Automatic stop and safety | TankLevel_LIT >= TankFull_SP (8.0 ft)<br>OR Estop_Cmd OR TankLevelHi_Alarm | RESET TankFillAuto_Cmd     |
| 4    | High-level alarm          | TankLevel_LIT >= TankHiAlarm_SP (9.0 ft)                                   | Energise TankLevelHi_Alarm |
| 5    | Low-level alarm           | TankLevel_LIT <= TankLoAlarm_SP (1.0 ft)                                   | Energise TankLevelLo_Alarm |

### How the whole tank-filling process works (in order):

| Situation                                         | What the program does                                                                                                             |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1. Level falls to 2.0 ft or below                 | Rung 2 SETs TankFillAuto_Cmd. Rung 1 energises TankValveOpen. The fill valve opens.                                               |
| 2. Tank is filling (level between 2.0 and 8.0 ft) | The SET latch holds TankFillAuto_Cmd ON even though the level is above 2.0 ft. The valve stays open.                              |
| 3. Level reaches 8.0 ft                           | Rung 3 RESETs TankFillAuto_Cmd. Rung 1 loses power and the valve closes.                                                          |
| 4. Liquid is used and level falls from 8.0 ft     | The command stays OFF and the valve stays closed until the level again reaches 2.0 ft. Then the cycle repeats.                    |
| 5. Level reaches 9.0 ft (abnormal)                | Rung 4 turns the high alarm ON (beacon and horn). Rung 3 resets the command. The negated contact in Rung 1 also blocks the valve. |
| 6. Level falls to 1.0 ft or below (abnormal)      | Rung 5 turns the low alarm ON. Filling is already running because the level is below 2.0 ft.                                      |
| 7. Emergency stop pressed at any time             | Rung 3 resets TankFillAuto_Cmd and the valve closes. It cannot restart while Estop_Cmd is TRUE.                                   |

**Note:** The fill starts at 2.0 ft but stops at 8.0 ft. This gap between the start and stop levels stops the valve from opening and closing again and again near a single level.

### 5.3 Evidence 3: Logic Test Table (Seven Scenarios)

**Question:** Show at least six test scenarios with conditions and expected outputs (Step 13).

**Method:** (E-stop means Estop\_Cmd.) For each scenario, I take the inputs, solve the rungs from top to bottom, and write the expected outputs. TankFillAuto\_Cmd "before" means its value at the start of the scan. The valve and alarm values are the final state.

| No. | Scenario                                 | Level (ft) | E-stop | Fill cmd before | Fill cmd after | Valve  | Hi alarm | Lo alarm |
|-----|------------------------------------------|------------|--------|-----------------|----------------|--------|----------|----------|
| 1   | Empty tank (fill start point)            | 2.0        | FALSE  | FALSE           | TRUE           | OPEN   | OFF      | OFF      |
| 2   | Normal filling                           | 5.0        | FALSE  | TRUE            | TRUE           | OPEN   | OFF      | OFF      |
| 3   | Full tank                                | 8.0        | FALSE  | TRUE            | FALSE          | CLOSED | OFF      | OFF      |
| 4   | High-level alarm                         | 9.0        | FALSE  | TRUE            | FALSE          | CLOSED | ON       | OFF      |
| 5   | Low-level alarm                          | 1.0        | FALSE  | FALSE           | TRUE           | OPEN   | OFF      | ON       |
| 6   | Emergency stop while filling             | 5.0        | TRUE   | TRUE            | FALSE          | CLOSED | OFF      | OFF      |
| 7   | Emergency stop when level is low (extra) | 1.5        | TRUE   | FALSE           | FALSE          | CLOSED | OFF      | OFF      |

**Note:** In Scenarios 3, 4 and 6 the valve closes one scan after the command is reset. A scan takes only a few milliseconds, so this is not noticeable.

#### Working for each scenario

**Scenario 1 (level 2.0 ft, Estop FALSE).** Rung 2:  $2.0 \leq 2.0$  is TRUE, so SET TankFillAuto\_Cmd = TRUE. Rung 3:  $2.0 \geq 8.0$  is FALSE, Estop is FALSE, Hi alarm is FALSE, so no reset. Rung 1: TRUE AND NOT FALSE = TRUE, so the valve is OPEN. Rung 4:  $2.0 \geq 9.0$  is FALSE. Rung 5:  $2.0 \leq 1.0$  is FALSE. **Result: filling starts, no alarm.**

**Scenario 2 (level 5.0 ft, command already TRUE).** Rung 2:  $5.0 \leq 2.0$  is FALSE, but the latch keeps the command TRUE. Rung 3:  $5.0 \geq 8.0$  is FALSE, Estop is FALSE, Hi alarm is FALSE, so no reset. Rung 1: valve OPEN. Rungs 4 and 5 are FALSE. **Result: normal filling continues.**

**Scenario 3 (level 8.0 ft, command TRUE).** Rung 3:  $8.0 \geq 8.0$  is TRUE, so RESET TankFillAuto\_Cmd = FALSE. Rung 1: TankFillAuto\_Cmd is FALSE, so the valve is CLOSED. Rung 4:  $8.0 \geq 9.0$  is FALSE. **Result: tank is full, filling stops, no alarm.**

**Scenario 4 (level 9.0 ft, command TRUE).** Rung 4:  $9.0 \geq 9.0$  is TRUE, so TankLevelHi\_Alarm = ON. Rung 3:  $9.0 \geq 8.0$  is TRUE and Hi alarm is TRUE, so RESET the command. Rung 1: the command is FALSE and NOT Hi alarm is FALSE, so the valve is CLOSED. **Result: high alarm ON, valve closed.**

**Scenario 5 (level 1.0 ft, command FALSE).** Rung 5:  $1.0 \leq 1.0$  is TRUE, so TankLevelLo\_Alarm = ON. Rung 2:  $1.0 \leq 2.0$  is TRUE, so SET the command. Rung 3: no reset condition. Rung 1: valve OPEN. **Result: low alarm ON and filling runs.**

**Scenario 6 (level 5.0 ft, Estop TRUE, command TRUE).** Rung 3: Estop\_Cmd is TRUE, so RESET the command = FALSE. Rung 1: valve CLOSED. Rungs 4 and 5 are FALSE.

**Result: emergency stop closes the valve and there is no alarm.**

**Scenario 7 (level 1.5 ft, Estop TRUE, command FALSE).** Rung 2:  $1.5 \leq 2.0$  is TRUE, so SET the command = TRUE. Rung 3 comes after Rung 2 and Estop\_Cmd is TRUE, so RESET the command = FALSE. The last action wins. Rung 1: valve CLOSED. Rung 5:  $1.5 \leq 1.0$  is FALSE, so no low alarm. **Result: even though the level is low, the emergency stop keeps the valve closed.**

## 5.4 Evidence 4: Saved Project Files

**Requirement:** Submit the exported / saved TankFill OpenPLC project files with the report.

**What I submit:**

| File / Item             | Description                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| This report (PDF)       | C11_26_ACIS_17362_LAB03_Building_Your_First_PLC_Program.pdf                                              |
| TankFill project folder | The saved OpenPLC project folder created in Step 1 (see Figure 14). Compress it to a ZIP file if needed. |
| Figure 13               | Save / compile result with no errors.                                                                    |

Figure 14 shows the saved project folder and its files. The project was saved after the final rung was built and checked.

## 5.5 Evidence 5: Safety Explanation (Why the Emergency Stop Overrides Automatic Control)

An emergency stop exists to protect people, equipment and the environment. Automatic control only looks at the level, so it cannot see other dangers such as a leak, a burst pipe, a worker in danger, or a valve that is not behaving as expected. Only a person can see these. If the automatic logic could keep filling when the E-stop is pressed, the E-stop would be useless.

In this program, Estop\_Cmd is in the RESET condition of Rung 3. Rung 3 is placed after the SET in Rung 2. So even when the level is low and Rung 2 tries to start the fill, the RESET comes last and wins (Scenario 7). The valve stays closed as long as the E-stop is pressed. Because the E-stop button is latched, it stays pressed until someone deliberately releases it, so the machine cannot restart by itself.

In a real plant, the emergency stop should also be hard-wired so that it cuts power to the valve independently of the PLC. The PLC logic here is a second layer of protection. It is not a replacement for a hard-wired safety circuit.

## 5.6 Knowledge Check

### 1. Why are BOOL and REAL appropriate data types for this project?

BOOL is suitable for discrete signals that have only two states, such as the valve command, the E-stop, the alarms and the fill latch. REAL is suitable for the tank level and

the setpoints, because they are measured in feet and need decimal values, such as 5.37 ft, so they can be compared with 2.0, 8.0, 1.0 and 9.0.

## **2. What is the purpose of a SET/RESET pair for TankFillAuto\_Cmd?**

The SET coil turns the command ON when the level falls to 2.0 ft, and the command stays ON (latches) after the level rises above 2.0 ft. The RESET coil turns it OFF at 8.0 ft, on emergency stop, or on high alarm. Together they work like a memory. They make the tank fill between 2.0 and 8.0 ft and stop the valve from opening and closing repeatedly around one level.

## **3. What should happen if the tank reaches 9 ft while the fill valve remains commanded open?**

The high-level alarm must turn ON (beacon and horn). The program must reset TankFillAuto\_Cmd (Rung 3) and the negated contact in Rung 1 must close the valve, so filling stops before the tank overflows. The operator should then check why the normal stop at 8.0 ft did not work, for example a faulty level transmitter or a valve that is stuck.

## **4. Why should PLC logic be validated in simulation before use with physical equipment?**

Mistakes in the logic can cause overflow, spills, damaged equipment or injury to people. Testing in simulation or on paper finds these errors safely and cheaply. It also allows abnormal cases, such as high alarm and emergency stop, to be tested without any real risk.

## **7. Conclusion**

---

In this lab I built my first PLC program. The TankFill program uses five rungs to fill the tank automatically between 2.0 ft and 8.0 ft, to raise a low alarm at 1.0 ft and a high alarm at 9.0 ft, and to stop filling at once when the emergency stop is pressed. I learned to use BOOL and REAL variables, normal and negated contacts, comparison expressions, and SET and RESET coils. I also learned that the order of the rungs matters, and that safety conditions must always have the last word.

## **8. Recommendations**

---

- Add a hard-wired emergency stop that cuts power to the valve independently of the PLC.
- Test all scenarios, including alarms and emergency stop, in simulation before connecting to real equipment or an HMI.
- Use clear variable names and short comments, and save the project regularly with a clean snapshot of the VM.
- Consider a small delay or deadband on the alarms in a real plant so that a level moving near a setpoint does not cause repeated false alarms.

## 9. References

---

International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab 03: Building Your First PLC Program (Student Edition). Intro to Critical Infrastructure for IT Learners. <https://www.icdfa.edu.ng>

International Electrotechnical Commission (IEC). IEC 61131-3: Programmable controllers, Part 3: Programming languages.

OpenPLC Project. OpenPLC Editor documentation. <https://openplcproject.com>

International Society of Automation (ISA). ANSI/ISA-5.1: Instrumentation Symbols and Identification.